Estructura de Datos

Tema 5:

Colas

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Cristian Rodríguez

Universidad
Rey Juan Carlos

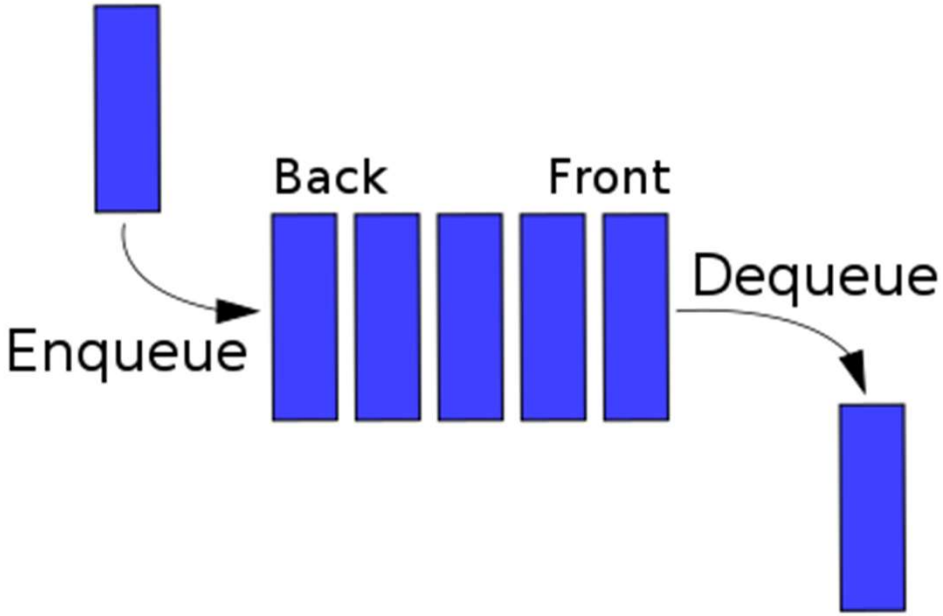
Curso 2023-2024

Contenidos

- Definición
- El TAD Cola: operaciones básicas
- Implementación
- Ejemplos de aplicación
- Colas y genericidad
- Ejercicios propuestos

Cola: definición

- Estructura de datos lineal, dinámica, que da soporte al concepto **FIFO**: **F**irst **I**n, **F**irst **O**ut

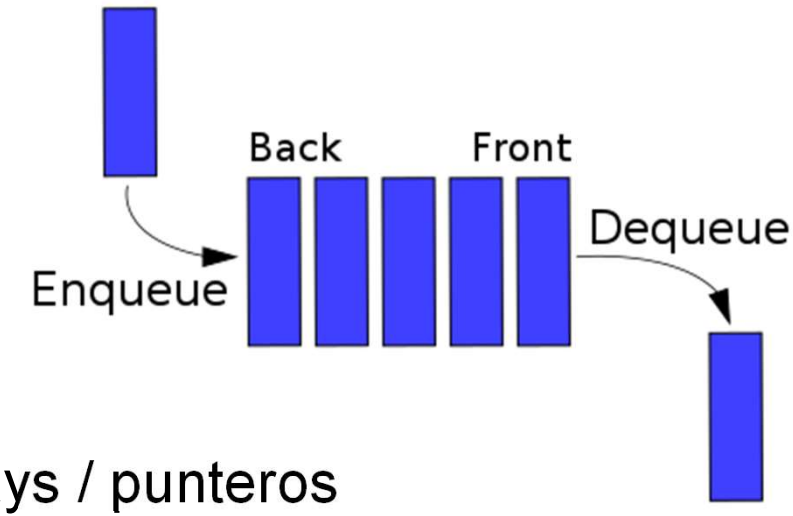


TAD Cola

- Independientemente de su implementación, una cola debe proporcionar al menos 5 operaciones básicas:
  - Meter un elemento (`enqueue`)
  - Sacar un elemento (`dequeue`)
  - Consultar si está vacía (`empty_queue`)
  - Conocer el primer y último elemento (`first`/`last`)
  - \+ Inicializar/crear

TAD Cola: especificación

- **Nombre:** Cola.
- **Valores posibles:** *vacía, otros (según elementos)*.
- **Operaciones:**
  - `enqueue`
  - `dequeue`
  - `first`
  - `last`
  - `empty_queue`
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)



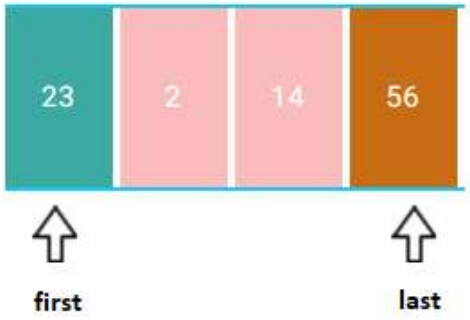
Implementación

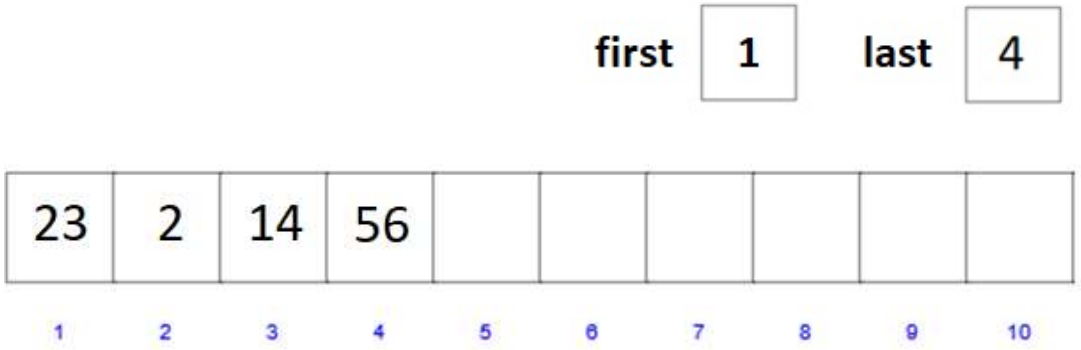
- Dos formas habituales de implementarla: con un array o utilizando memoria dinámica.
  - Externamente no debería ser posible distinguirlo
- Utilizando arrays hay limitaciones de espacio y otros inconvenientes.
- Utilizando punteros y memoria dinámica no hay límite de espacio (en teoría) y la implementación es más eficiente.

Implementación mediante arrays

Implementación mediante arrays

- Necesitaremos dos variables enteras para indicar la posición del primer y último elemento de la cola + un array del tipo base a almacenar





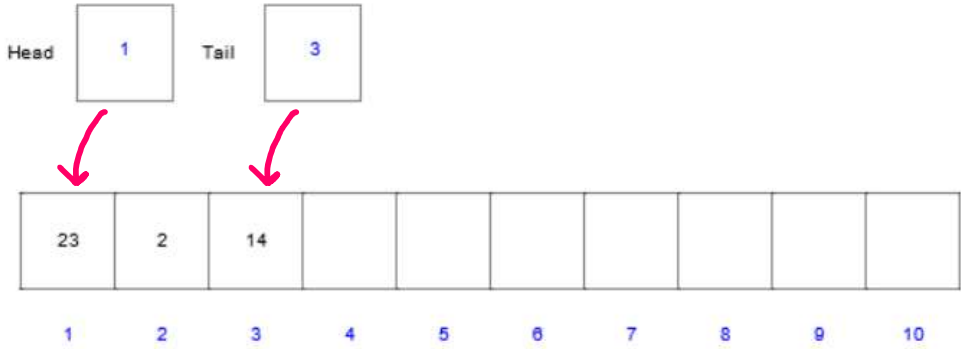
Implementación mediante arrays

- Hay muchas implementaciones posibles, algunas muy sencillas, otras más complicadas, pero todas peores en términos de eficiencia (y a menudo de complejidad de implementación) que las implementaciones con punteros.
  a) Mediante un array lineal con eliminación simple
  b) Mediante un array lineal con eliminación reorganizada
  c) Mediante un array circular con cuenta de longitud
  d) Mediante un array circular con posición libre
  e) Etc.

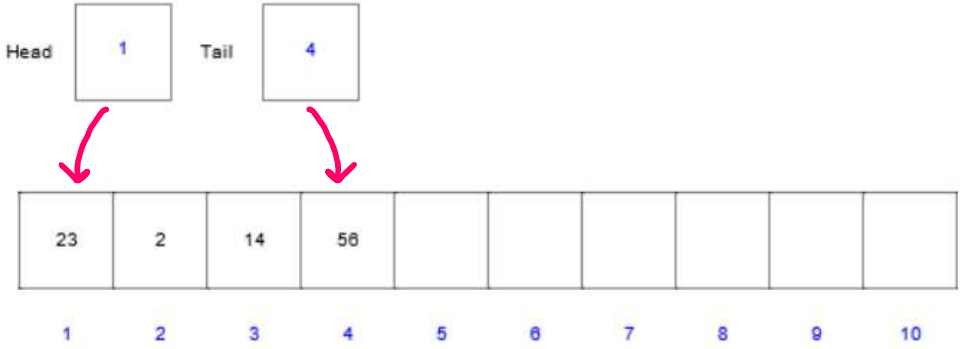
(a) Array lineal con eliminación simple

- La inserción se produce siempre en el primer hueco disponible (por la parte de atrás):

(I)



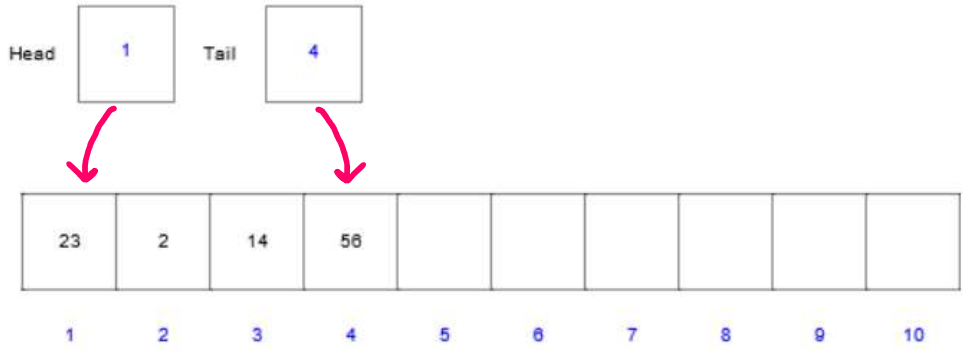
(II)



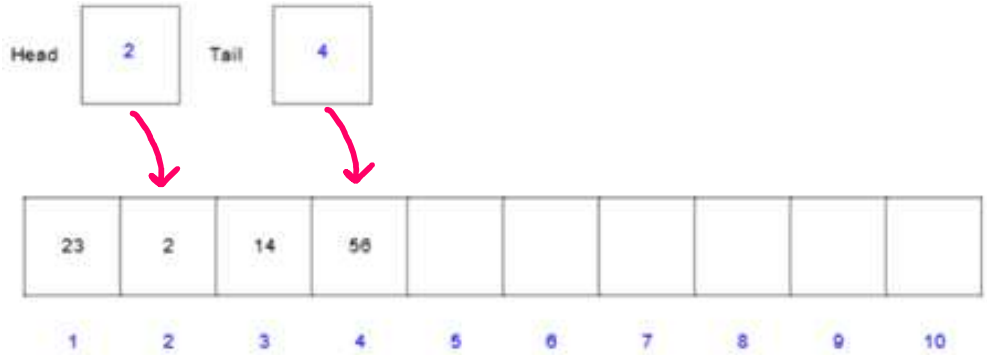
(a) Array lineal con eliminación simple

- Sacar un elemento consiste en mover el índice del primer elemento al siguiente (`first := first + 1`):

(I)



(II)



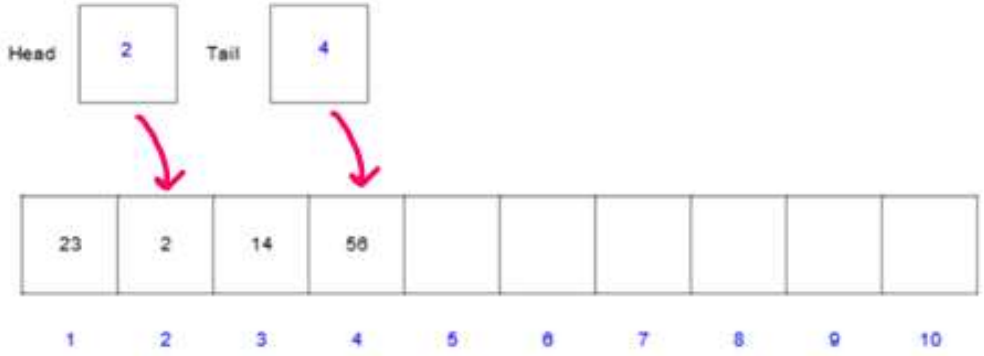
(a) Array lineal con eliminación simple

- Esta implementación es muy sencilla, pero…
- La cola tiene un tamaño prefijado (rígido)
- Sacar un elemento es muy sencillo y eficiente (O(1)) pero deja memoria no utilizable lo cual implica:
  - La necesidad de reorganizarla en algún momento
  - El riesgo de no hacerlo y quedarse sin espacio cuando realmente sí lo hay

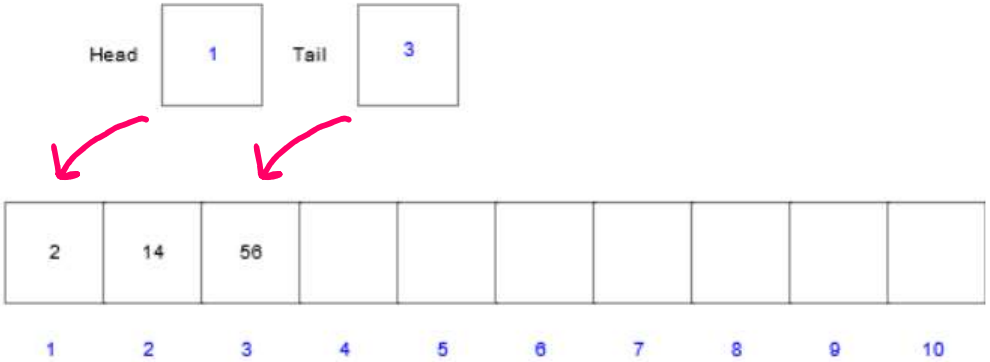
(b) Array lineal con reorganización

- Al sacar un elemento de la cola se reorganizan los elementos del array, situando el primero en la posición 1, etc.

(I)



(II)



(b) Array lineal con reorganización

- La implementación incluye una operación de borrado no tan simple de implementar, si bien no hace falta “front”:

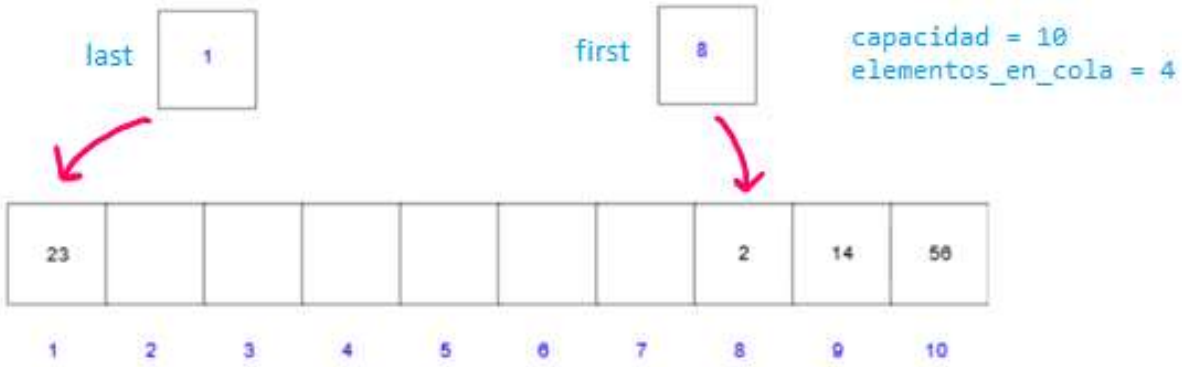
```
if (not empty_queue(cola)) then
  for i:=1 to tail-1
    cola[i] = cola[i + 1];
tail:= tail-1
```

→ O(n)

- La complejidad del borrado pasa a ser O(n) frente a la complejidad constante de la implementación (a)

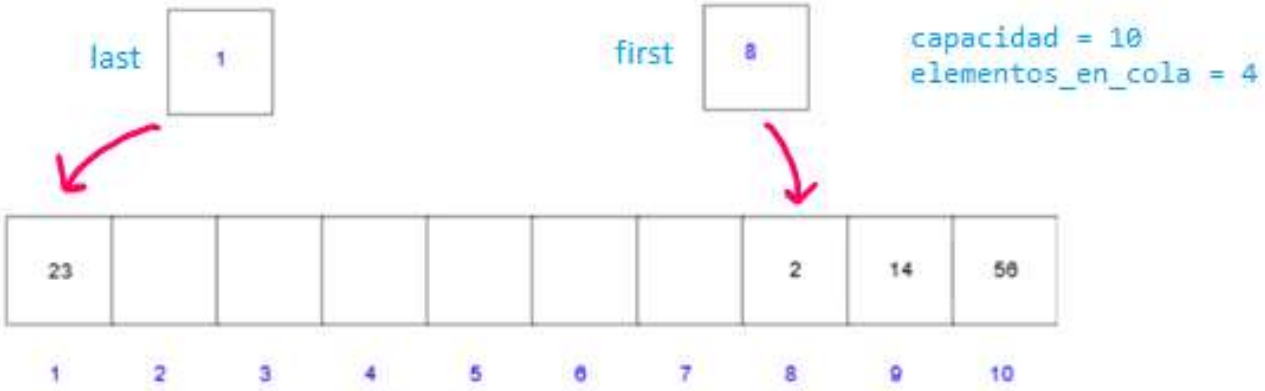
(c) Array circular

- La inserción es simple: avanzar el índice `last` a la siguiente posición y almacenar el dato: `new_last = (last mod 10) + 1`
- La eliminación no reorganiza: simplemente se avanza `first` a la siguiente posición: `new_first = (first mod 10) + 1`
- El array se diseña para continuar insertando en el hueco que queda entre `first` y `last`



(c) Array circular

- Para saber si la cola está vacía la implementación más sencilla es un entero `elementos_en_cola`: si cero → vacía.
- Cola llena: si `elementos_en_cola` es igual a la capacidad.
- Otra implementación deja un elemento vacío en el array para separar los índices `last` y `first`.



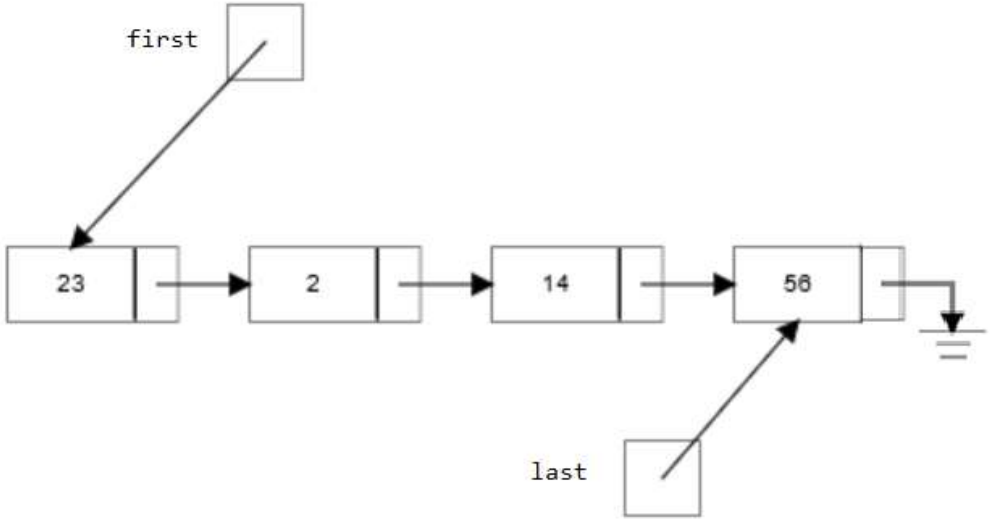
Cola como array (resumen)

- **Desventajas**:
  - Es necesario conocer el tamaño a priori.
  - La eliminación es poco eficiente en implementación lineal, no en el array circular
  - La implementación no es tan inmediata como en la pila
- Es necesario establecer **precondiciones** en varias operaciones cuya implementación más lógica sería utilizar excepciones
  - Es necesaria una reflexión diferente para cada tipo base.

Implementación en memoria dinámica

Implementación con memoria dinámica

- Utilizamos una estructura con dos punteros: `first` y `last`
- No hay máximo de elementos
- Inicialmente `first` y `last` están a NIL
- Antes de eliminar un elemento hay que comprobar si es posible
- La inserción y borrado tienen complejidad O(1): muy eficiente.



Implementación con memoria dinámica

- Utilizamos una estructura nodo

```
type
    nodo = record
        info: integer;
        sig: ^nodo;
    end;
```

- Y luego una estructura con los dos manejadores de la cola:

```
    cola = record
        first, last: ^nodo;
    end;
```

Implementación con memoria dinámica

```
type
  nodo = record
    info: integer;
    sig: ^nodo;
  end;
  tcola = record
    first, last: ^nodo
  end;
```

Contrato

```
procedure initialize_queue(var c: tcola);
function empty_queue(c: tcola): boolean;
function first(c: tcola): integer;
function last(c: tcola): integer;
procedure enqueue(var c: tcola; x: integer);
procedure dequeue(var c: tcola);
```

Implementación con memoria dinámica

```
procedure initialize_queue(var c: tcola);
begin
  c.first := nil;
  c.last := nil;
end;

function empty_queue(c: tcola): boolean;
begin
  empty_queue:= (c.first = nil) and (c.last = nil);
end;

function first(c: tcola): integer;
begin
  first:= c.first^.info;
end;

function last(c: tcola): integer;
begin
  last:= c.last^.info;
end;
```

Implementación memoria dinámica

```
procedure enqueue(var c: tcola; x: integer);
var
  new_node: ^nodo;
begin
  new(new_node);
  new_node^.info:= x;
  new_node^.sig:= nil;
  if empty_queue(c) then
    c.first:=new_node
  else
    c.last^.sig:=new_node;
  c.last:= new_node;
end;
```

Implementación memoria dinámica

```
procedure dequeue(var c: tcola);
var
  aux: ^nodo;
begin
  if not empty_queue(c) then
  begin
  aux:= c.first;
    c.first := c.first^.sig;
    if c.first = nil then c.last:= nil;
    dispose(aux);
  end;
end;
```

Funcionalidades adicionales

- Podemos ampliar nuestro TAD Cola con algunas operaciones extra tales como:
  - `clear()` → Vacía la cola
  - `show()` → Muestra todos los elementos
  - `num_elems()` → Cuenta el número de elementos de la cola
  - `copy()` → Genera un duplicado de la cola

Funcionalidades adicionales

```
procedure show_queue(c: tcola);
var
  aux: ^nodo;
begin
  if empty_queue(c) then writeln('Empty queue: No elements to show')
  else
  begin
    aux:= c.first;
    write('First->');
    while aux<>nil do
    begin
      write(aux^.info,'-');
      aux:= aux^.sig;
    end;
    writeln('>Last');
  end
end;

procedure clear_queue(var c: tcola);
begin
  while not empty_queue(c) do dequeue(c);
end;
```

Funcionalidades adicionales

```
function num_elems_queue(c: tcola): integer;
var
  aux: ^nodo;
  count: integer;
begin
  count:= 0;
  aux:= c.first;
  while aux<>nil do
  begin
      count:=count + 1;
      aux:= aux^.sig
  end;
  num_elems_queue:= count
end;
```

Funcionalidades adicionales

```
procedure copy_queue(c: tcola; var c2: tcola);
var
  aux: ^nodo;
begin
  initialize_queue(c2);
  if not empty_queue(c) then
  begin
    aux:= c.first;
    while aux<>nil do
    begin
      enqueue(c2,aux^.info);
      aux:= aux^.sig
    end;
  end
end;
```

Aplicaciones

- Muy utilizadas en computación para un gran número de aplicaciones tales como:
  - Gestión de tareas de la CPU
  - Organización de trabajos de impresión
  - Gestion de dispositivos y recursos compartidos
  - Gestión de centros de llamadas
  - Almacenamiento en búfer
  - Etc.

Complejidad

- Todas las operaciones básicas de las colas son de complejidad O(1)
  - initialize_queue
  - enqueue
  - dequeue
  - first/last
  - empty_queue
- Todas las operaciones aquí incluidas como adicionales son dependientes del número de elementos → O(n)
  - show_queue
  - clear
  - copy_queue
  - num_elems_queue

Colas y genericidad

Limitaciones del enfoque anterior

- Hasta ahora hemos visto una aplicación específica de las colas: cola de enteros
- Al igual que con las pilas, las colas de otros elementos tienen idéntico comportamiento
- Programaremos colas genéricas en función de un elemento Telemento que debe implementar…
  - `assign(var e: TElemento; e2: TElemento);`
  - `lessThan(e: TElemento; e2: TElemento): boolean;`
  - `equalTo(e: TElemento; e2: TElemento): boolean;`
  - …

Unit Cola genérica

- Implementar una unidad ColaGenerica en función de Telemento
  - … se propone como ejercicio ☺

Ejercicios propuestos

Ejercicio propuesto 1

Una empresa guarda un listado de clientes en una cola de registros [cod_cliente, activo] donde el campo “activo” indica si el cliente sigue trabajando o no con la empresa.

Implementar un subprograma que elimine de la cola aquellos clientes marcados como no activos.

No es necesario leer la entrada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(n)

Ejercicio propuesto 2

Programa un procedimiento que muestre por pantalla la información de una cola que recibe como parámetro en orden inverso al de inserción.

No es necesario leer la entrada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(n)

Ejercicio propuesto 3

Programa una función que reciba una cola con números enteros distintos en un rango y retorne el elemento que falta, o -1 si la secuencia es completa (incluye todos los enteros entre el menor y el mayor observados).

- Ejemplo 1: c = {10,12,13,15,11}, salida: {14} (menor=10, mayor=15)
- Ejemplo 2: c = {6,1,2,7,4,5,3}, salida: {} (menor=1, mayor=7)

No es necesario leer la entrada ni imprimir nada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(1)

Ejercicio propuesto 4

- Programa una función que reciba una cola que contiene números enteros en el rango de 1 a n y retorne una estructura de datos con todos los números que aparecen más de una vez y su frecuencia.
- Si no se repite ninguno, retornar una estructura vacía.
  - Ejemplo 1: n = 4 , c = {1,1,4,3,1,4,2}, salida: {[1,3], [4,2]}
  - Ejemplo 2: n = 5, c = {2,3,1,2,5}, salida: {[2,2]}

No es necesario leer la entrada ni imprimir nada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(n)

Ejercicio propuesto 5 (reto)

Programa un subprograma que reciba una pila y una cola de enteros e intercambie sus elementos sin alterar el orden, poniendo todos los elementos de la pila en la cola y todos los de la cola en la pila sin modificar el orden.

- Ejemplo 1: p={4,3,2,1}, c={8,7,6,5}, salida: c={4,3,2,1}, p={8,7,6,5}
- Ejemplo 2: p={2,1}, c={0,1}, salida: c={2,1}, p={0,1}

No es necesario leer la entrada ni imprimir nada

Complejidad de tiempo esperada: O(n)

Espacio auxiliar esperado: O(1)